Лекция 6. Агентные модели и онтологии в мультиагентных робототехнических системах

Архитектуры агентов, дескрипционные логики, OWL, FIPA ACL, KnowRob и IEEE 1872

90 минут · две части по 45

Курс «Управление распределёнными робототехническими системами»

План лекции

1. Агент: свойства, компоненты, формальная модель
2. Реактивные архитектуры и субсумпция Брукса
3. Делиберативные агенты и модель BDI
4. Гибридные архитектуры и обучающиеся агенты
5. Онтология: определение Грубера и компоненты
6. Дескрипционные логики, RDF, RDFS, OWL 2
7. Речевые акты, KQML и FIPA ACL
8. Contract Net, платформы JADE и SPADE
9. KnowRob, IEEE 1872 (CORA), RoboEarth
10. Сквозной пример: складская онтология и диалог агентов

Синие — первая часть · янтарные — вторая часть

Часть 1. Агент и его архитектуры

Что такое агент формально, и три исторических ответа на вопрос, как устроить его внутренности: реактивный, делиберативный и гибридный.

1 Зачем это нужно: склад с роботами разных вендоров

– MAPC — совокупность автономных агентов, взаимодействующих со средой и друг с другом

– Эффективность зависит не только от качества агента, но и от взаимопонимания

– Робот A знает «зону зарядки», робот B — только «точку 14»: интеграция ломается

– Два ответа: внутренняя организация агента и общая модель мира

– Онтология и язык коммуникации — это инфраструктура понимания, а не украшение

1 Агент: определение и четыре свойства

– Агент воспринимает среду сенсорами и действует актуаторами ради своих целей

– **Автономность** — действует без прямого вмешательства, сам управляет состоянием

– **Реактивность** — своевременно отвечает на изменения среды

– **Проактивность** — инициирует действия ради цели, а не только реагирует

– **Социальность** — взаимодействует с другими через язык коммуникации

Ключевая идея

«Слабое понятие агентности» (Вулдридж, Дженнингс) — эти четыре свойства. «Сильное» добавляет ментальные категории: знания, убеждения, желания, намерения — и ведёт прямо к архитектуре BDI.

1 Компоненты агента

– **Сенсоры**: камеры, лидары, дальномеры, ИИМ, GNSS — поток зашумлён и неполон

– **Актуаторы**: приводы, манипуляторы, захваты, а также коммуникационные интерфейсы

– **Контроллеры**: от жёсткой логики микроконтроллера до планировщиков и нейросетей

– **База знаний**: карты, модели объектов, правила, модели других агентов, задачи

– База знаний обновляется в работе — из собственного опыта и от других агентов

1 Формальная модель агента

$$\mathrm{Ag} : \mathrm{Per}^* \to \mathrm{Ac}, \quad next : I \times \mathrm{Per} \to I, \quad action : I \to \mathrm{Ac}$$

– Чисто реактивный агент: action: Per → Ac — решение по последнему перцепту

– Агент с внутренним состоянием I аппроксимирует зависимость от всей истории

– Среда: функция переходов τ: E × Ac → 2^E — недетерминизм заложен явно

– Задача проектирования: истории h = $e_0 \to^{\alpha_0} e_1 \to^{\alpha_1} \ldots$ удовлетворяют критерию u: E → $\mathbb{R}$

Per* — конечные последовательности перцептов; выбор Ag определяет множество порождаемых историй.

? Как думаете?

Автономный пылесос строит карту квартиры, объезжает препятствия и сам едет на базу заряжаться. Является ли он агентом мультиагентной системы?

- **A** Да — он автономен, реактивен и проактивен
- **B** Нет — не хватает социальности
- **C** Да, если у него есть Wi-Fi
- **D** Нет — у него нет символьной модели мира

? Как думаете?

Автономный пылесос строит карту квартиры, объезжает препятствия и сам едет на базу заряжаться. Является ли он агентом мультиагентной системы?

- **A** Да — он автономен, реактивен и проактивен
- **B** Нет — не хватает социальности
- **C** Да, если у него есть Wi-Fi
- **D** Нет — у него нет символьной модели мира

ОТВЕТ

Нет — не хватает социальности. Все три остальных свойства есть, но без языка взаимодействия с другими агентами он остаётся роботом-одиночкой.

2 Три класса архитектур

– **Архитектура** — организация внутренних компонентов и потоков между ними

– **Реактивные**: правила «условие → действие», без модели мира и рассуждений

– **Делиберативные**: явная символьная модель мира, планирование, вывод

– **Гибридные**: многоуровневые, реактивный низ и планирующий верх

– Обучающиеся агенты — не четвёртый класс, а контур адаптации поверх любого

2 Реактивная архитектура: бунт Брукса (1986)

– Правила «условие → действие», прямая связь сенсоров с актуаторами

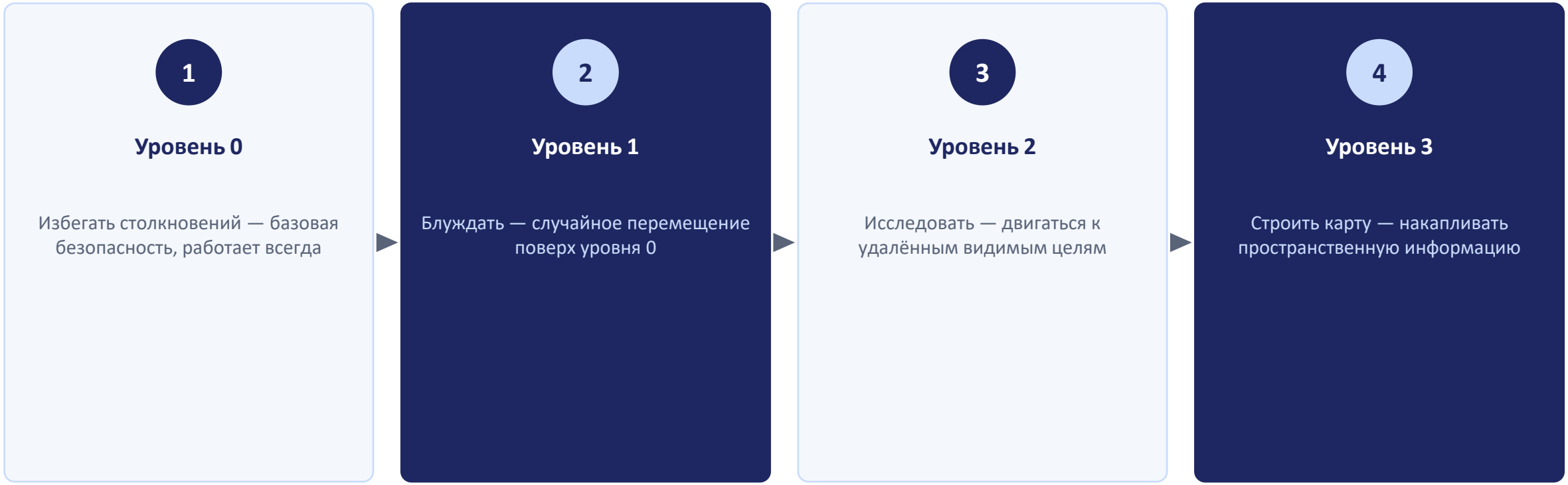
– Тезис 1: интеллектуальное поведение возможно без символьного представления

– Тезис 2: интеллект эмерджентен — рождается из взаимодействия простых поведений со средой

– Время реакции — миллисекунды, вычислительные требования низкие

– Робастность: нет модели мира — нечему устаревать и ломаться

2 Субсумпционная архитектура: уровни компетенции



Уровни работают параллельно и асинхронно, каждый связан с сенсорами и актуаторами напрямую.

2 Субсумпция: подавление, замещение и цена простоты

– **Подавление (suppress)**: верхний уровень подменяет вход нижнего

– **Замещение (inhibit)**: верхний уровень блокирует выход нижнего

– Нижние уровни продолжают работать при отказе верхних — деградация мягкая

– Минус: агент «живёт настоящим», долгосрочное планирование невозможно

– Минус: спроектировать нужное эмерджентное поведение всё труднее с ростом числа уровней

Факт

Идея субсумпции жива: в современных стеках ROS её роль играют слои безопасности, которые перехватывают команды скорости от планировщика при срабатывании бампера или лидарного стоп-поля.

“

Мир — лучшая собственная модель. Зачем хранить копию среды в памяти, если можно всё время смотреть на саму среду?

Родни Брукс, программная идея субсумпционной архитектуры (1986)

3 Делиберативный агент: думать перед тем, как ехать

– Явная символьная модель мира обновляется по сенсорным данным

– По целям строится план — последовательность действий до целевого состояния

– План исполняется с контролем результата и перепланированием

– Сильные стороны: предвидение, оптимизация маршрутов и последовательностей

– Плата: планирование экспоненциально сложно, модель устаревает быстрее цикла рассуждений

3 BDI: убеждения, желания, намерения

– **Beliefs (B)** — информационное состояние: что агент считает истинным, возможно, ошибочно

– **Desires (D)** — мотивационное состояние: чего хотел бы достичь; могут противоречить

– **Intentions (I)** — деливеративное состояние: на что выделены ресурсы и приняты обязательства

– Состояние агента — тройка $\langle B, D, I \rangle$, где $B \subseteq Bel$, $D \subseteq Des$, $I \subseteq Int$

– Намерения инерционны: не пересматриваются на каждом шаге

Ключевая идея

Инерция намерений — это и есть баланс реактивности и целенаправленности. Агент, пересматривающий цели каждый такт, не доедет никуда; агент, не пересматривающий их никогда, поедет в стену.

3 Архитектура BDI-агента



Beliefs — что есть **Desires — что хотелось бы** **Intentions — что делаем**

Сенсоры питают пересмотр убеждений (brf); убеждения и текущие намерения порождают варианты-желания; фильтр выбирает намерения, библиотека планов даёт способ их достичь, действие замыкает контур через среду.

3 Цикл практического рассуждения BDI



Внутренний цикл прерывается, если намерение достигнуто, стало невозможным или план перестал быть корректным.

3 Дилемма reconsider и практические реализации

– **Смелый агент** редко пересматривает намерения: эффективен в статике, негибок в динамике

– **Осторожный агент** пересматривает часто: адекватен изменчивой среде, но тратит ресурсы

– Оптимум зависит от темпа изменений среды — это проектное решение, а не константа

– PRS, AgentSpeak/Jason, JACK: библиотеки готовых планов вместо планирования «с нуля»

– Готовые планы делают BDI применимым в реальном времени

? Как думаете?

Складской AGV работает в цеху, где люди и погрузчики движутся постоянно. Какую стратегию пересмотра намерений выбрать?

- **A** Смелую — реже думать, больше везти
- **B** Осторожную — среда динамична
- **C** Отказаться от намерений вовсе, оставить реактивность
- **D** Пересматривать строго раз в секунду по таймеру

? Как думаете?

Складской AGV работает в цеху, где люди и погрузчики движутся постоянно. Какую стратегию пересмотра намерений выбрать?

- **A** Смелую — реже думать, больше везти
- **B** Осторожную — среда динамична
- **C** Отказаться от намерений вовсе, оставить реактивность
- **D** Пересматривать строго раз в секунду по таймеру

ОТВЕТ

Осторожную: среда меняется быстрее, чем выполняется план. Но пересмотр разумно запускать по событиям, а не по таймеру — иначе агент думает вхолостую.

4 Гибридные архитектуры: две схемы

Горизонтальная (TouringMachines)

- Все уровни параллельно у сенсоров и актуаторов
- Реактивный, планирующий, моделирующий
- Медиатор разрешает конфликты предложений
- Плюс: быстрый отклик на любом уровне
- Минус: медиатор — узкое место и источник сложности

Вертикальная (InteRRaP)

- Сенсорный поток проходит уровни последовательно
- Поведенческий, локального планирования, кооперативный
- Восходящая активация, нисходящее исполнение
- Плюс: понятная ответственность уровней
- Минус: отказ уровня рвёт всю цепочку

4 InteRRaP и обучающиеся агенты

– **Поведенческий уровень** — модель мира, реактивные поведения, быстрый отклик

– **Уровень локального планирования** — база планов, целенаправленное поведение агента

– **Кооперативный уровень** — социальная база знаний: модели других агентов, протоколы

– Каждому уровню соответствует свой слой базы знаний

– Обучение — контур адаптации: критик, обучающий элемент, генератор проблем

Ключевая идея

InteRRaP явно встраивает социальное взаимодействие в архитектуру отдельного агента — именно поэтому она особенно интересна для MAPC. Кооперация здесь не надстройка, а штатный уровень.

4 Сравнение архитектур агентов

Критерий	Реактивная (Брукс)	Делиберативная (BDI)	Гибридная (InteRRaP)
Модель мира	отсутствует	явная символьная	многоуровневая
Время реакции	миллисекунды	секунды и более	зависит от уровня
Долгосрочное планирование	нет	да	да (верхние уровни)
Вычислительные требования	низкие	высокие	средние и высокие
Предсказуемость	эмерджентная	объяснима через B, D, I	смешанная
Динамичные среды	высокая пригодность	ограниченная	высокая
Кооперация	косвенная (стигмергия)	через коммуникацию	встроенный уровень

Таблица 6.1 пособия. Обучающиеся агенты — не отдельная колонка, а контур адаптации поверх любой из трёх схем.

Конец первой части

Разобрали агента изнутри: формальную модель, реактивную субсумпцию, BDI и гибриды. После перерыва — как агенты договариваются: онтологии, дескрипционные логики, FIPA ACL и складской пример.

Часть 2. Онтологии и коммуникация агентов

Как формально описать общее понимание мира, как на нём строится вывод и как агенты обмениваются сообщениями со ссылкой на онтологию.

5 Онтология: определение Грубера

– Термин заимствован из философии, где означает учение о бытии

– **Концептуализация** — абстрактная модель фрагмента мира: объекты, понятия, отношения

– **Явная спецификация** — формальное машиночитаемое описание этой модели

– Уточнения Борста и Штудера: формальная явная спецификация **разделяемой** концептуализации

– Разделяемость — это согласованность сообществом, а не просто наличие файла

Определение

Онтология (Т. Грубер, 1993) — явная спецификация концептуализации: an explicit specification of a conceptualization.

5 Компоненты онтологии и её роль в МАРС

– **Концепты (классы)**: Робот, Сенсор, Задача транспортировки

– **Отношения**: таксономические, партономические, функциональные (исполняет, расположен в)

– **Атрибуты**: грузоподъёмность, заряд батареи; **аксиомы**: ограничения интерпретации

– **Экземпляры**: agv_07, стеллаж rack_A3 — фактологическая составляющая

– Три задачи в МАРС: единая терминология, логический вывод, интероперабельность

6 Дескрипционные логики: TBox и ABox

$$AGVRobot \equiv \text{Robot} \sqcap \exists hasSensor.\text{Lidar} \sqcap \exists performs.\text{WarehouseTask}$$

– Конструкторы: C ⊓ D, C ⊔ D, ¬C, ∃R.C, ∀R.C, ≥n R.C, ≤n R.C

– **TBox**: включения C ⊑ D и определения C ≡ D — терминология

– **ABox**: AGVRobot(agv_07), locatedIn(agv_07, zone_A) — факты об индивидах

– DL — семейство разрешимых фрагментов логики первого порядка

Читается: складской робот — это робот, имеющий лидар и выполняющий хотя бы одну складскую задачу.

6 Что делает ризонер

– **Непротиворечивость**: существует ли вообще модель, удовлетворяющая базе знаний

– **Классификация**: построение полной иерархии концептов из аксиом

– **Проверка принадлежности**: относится ли индивид a к концепту C

– Реализации: HermiT, Pellet, FaCT++ — подключаются прямо из Protégé

– Наличие ризонеров отличает онтологию от произвольной модели данных

Пример

Робот с разряженной батареей, находящийся вне зоны зарядки, автоматически попадает в концепт «требующий вмешательства» — этот факт нигде не записан, он выведен из аксиом.

6 Стек W3C: RDF, RDFS, OWL 2, SPARQL

– **RDF**: триплеты «субъект — предикат — объект», ресурсы с URI, граф знаний

– **RDFS**: rdfs:Class, rdfs:subClassOf, rdfs:subPropertyOf, rdfs:domain, rdfs:range

– **OWL 2**: эквивалентность и дизъюнктность, транзитивность, кардинальность, цепочки свойств

– Профили OWL 2: **EL** (полиномиальный вывод), **QL** (доступ к БД), **RL** (правила)

– Запросы — SPARQL; инструменты — Protégé, Apache Jena, RDFLib и Owlready2

6 Как строят онтологию



Процесс итеративный: тестовые вопросы из шага 1 служат приёмочными тестами на шаге 5.

? Как думаете?

В онтологии склада записано StorageZone owl:disjointWith ChargingZone, а в фактах zone_A объявлена и зоной хранения, и зоной зарядки. Что скажет ризонер?

A Ничего, просто проигнорирует один из фактов

B Отнесёт zone_A к обоим классам

C Объявит базу знаний противоречивой

D Создаст новый общий подкласс

? Как думаете?

В онтологии склада записано StorageZone owl:disjointWith ChargingZone, а в фактах zone_A объявлена и зоной хранения, и зоной зарядки. Что скажет ризонер?

- **A** Ничего, просто проигнорирует один из фактов
- **B** Отнесёт zone_A к обоим классам
- **C** Объявит базу знаний противоречивой
- **D** Создаст новый общий подкласс

ОТВЕТ

Объявит базу знаний противоречивой: аксиома дизъюнктности запрещает пересечение, и ризонер обязан сообщить о несовместимости фактов и терминологии.

7 Речевые акты и язык KQML

– Теория речевых актов (Остин, Сёрл): высказывание есть действие, меняющее состояние мира

– **Перформатив** — типизированный коммуникативный акт: сообщить, запросить, предложить

– KQML (DARPA Knowledge Sharing Effort, начало 1990-х): tell, ask-if, achieve, subscribe

– Три уровня сообщения: содержательный, коммуникативный, уровень сообщения

– Минусы KQML: нестрогая семантика перформативов, нет формальных протоколов

7 FIPA ACL: структура сообщения

– Обязательный перформатив плюс параметры: sender, receiver, content

– language — язык содержания; **ontology** — модель, в терминах которой его читать

– protocol и conversation-id — привязка к схеме диалога и связывание сообщений

– reply-with / in-reply-to — сцепка вопросов и ответов; reply-by — дедлайн

– Семантика 22 перформативов формально задана на языке SL

Ключевая идея

Онтология вынесена в явный параметр сообщения. Получатель знает не только текст, но и словарь, в котором его интерпретировать, — здесь смыкаются обе половины лекции.

7 Основные перформативы FIPA ACL

Перформатив	Класс акта	Назначение
inform	передача информации	сообщить, что утверждение истинно
request	побуждение	попросить выполнить действие
query-if / query-ref	запрос информации	спросить об истинности / о значении
agree / refuse	ответ на запрос	принять / отклонить просьбу
cfp	переговоры	объявить приём предложений
propose	переговоры	выдвинуть предложение с условиями
accept-proposal / reject-proposal	переговоры	принять / отклонить предложение
failure / inform-done	отчёт	сообщить о неуспехе / о выполнении

Всего 22 перформатива; отдельно стоят subscribe (подписка на изменения) и not-understood (управление диалогом).

8 Протокол FIPA Contract Net



Р. Смит, 1980; стандартизован FIPA. Основной механизм рыночного распределения задач между роботами.

8 Платформы JADE и SPADE

– **JADE** (Java, Telecom Italia): контейнеры агентов на разных узлах, распределённое исполнение

– Обязательные службы FIPA: **AMS** (жизненный цикл) и **DF** (жёлтые страницы по услугам)

– Класс Ontology и кодеки: Java-объекты автоматически становятся содержанием ACL-сообщений

– **SPADE** (Python): транспорт XMPP, агент — клиент вида name@server, удобно в облаке

– Типовая схема MAPC: низкий уровень — ROS, социальный — JADE или SPADE через мост

9 KnowRob: знания на борту робота

– Тенорт и Беец: OWL-онтология + вывод на Prolog, интеграция с ROS

– **Вычисляемые предикаты**: ответы не только по фактам, но и по восприятию и геометрии

– Описывает объекты и их функции, действия и их декомпозицию, пространственные отношения

– Описывает и самого робота: кинематику и собственные возможности

– KnowRob 2.0: журналы эпизодов — рассуждение о собственном прошлом опыте

Кейс

Команда «накрой на стол» не содержит списка предметов. Робот выводит через онтологию, какие объекты нужны, где они обычно лежат и какими действиями достигается результат.

9 IEEE 1872 (CORA), RoboEarth, OntoRob

– Рост числа несовместимых частных онтологий потребовал стандартизации

– **IEEE 1872-2015**: базовая онтология CORA на верхнеуровневой SUMO, рабочая группа ORA

– CORA определяет: робот, робототехническая система, группа роботов, среда, пакет POS (позиция, ориентация, поза)

– **RoboEarth** (2009–2014): облачный обмен моделями объектов, картами и рецептами действий

– Развитие: IEEE 1872.2 (автономная робототехника), фреймворк Rapyuta, OntoRob для модулей

9 Экосистема в цифрах

1993

определение онтологии Т. Грубера

22

перформатива в стандарте FIPA ACL

3

профиля OWL 2: EL, QL, RL

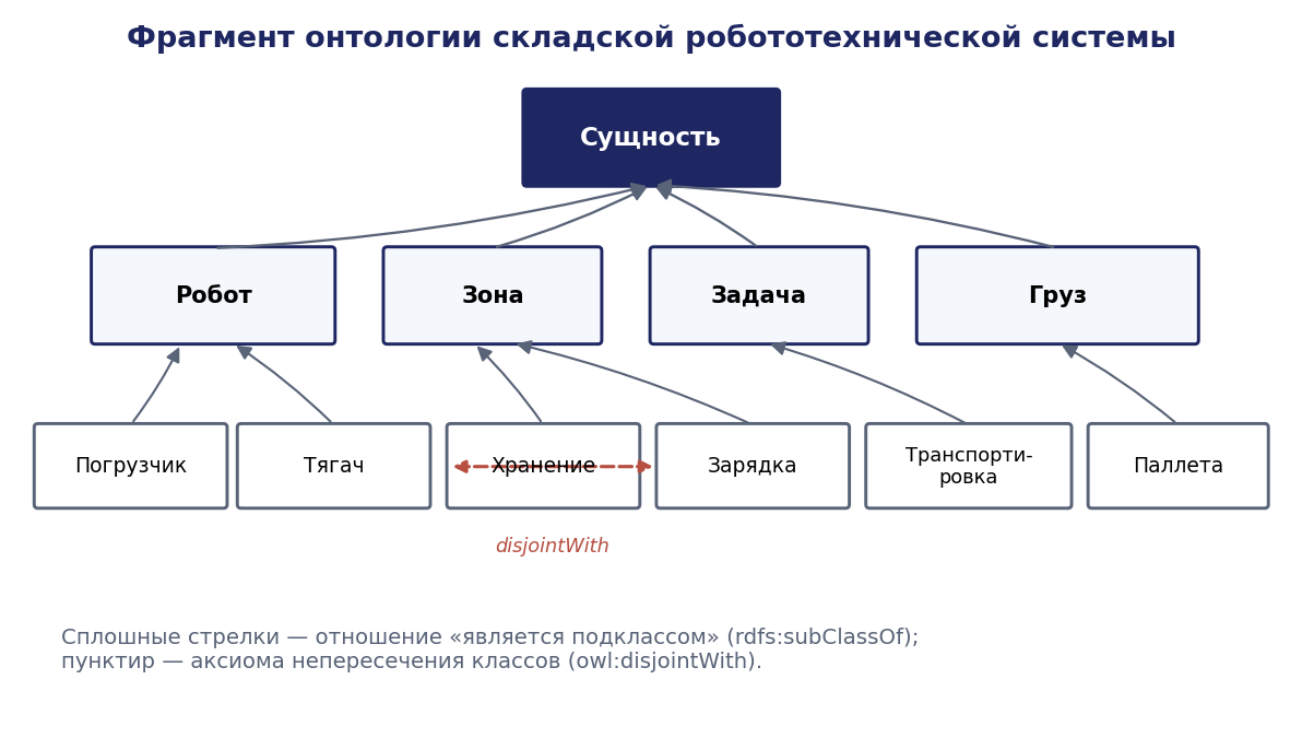
2015

IEEE 1872 — базовая онтология CORA

– Область молодая по меркам стандартизации, но опорные документы уже устоялись

– Между определением Грубера и промышленным стандартом прошло чуть больше 20 лет

10 Фрагмент онтологии складской РТС



Робот и его виды **Зоны и их аксиомы** **Задачи и грузы**

Корневой класс «Сущность» и четыре ветви: Робот, Зона, Задача, Груз с конкретизациями (Погрузчик, Тягач, Хранение, Зарядка, Транспортировка, Паллета). Пунктиром показана аксиома owl:disjointWith между зонами хранения и зарядки.

10 Складская онтология: TBox, ABox и вывод

– TBox: Robot ⊐ AGV ⊐ ForkliftAGV; Zone ⊐ StorageZone, ChargingZone; Task, Pallet

– Свойства: locatedIn, assignedTo, hasPayload (кг), hasMass (кг), batteryLevel (%)

– ABox: agv_07 — ForkliftAGV, payload 1200 кг, заряд 84 %, zone_A; pallet_512 — 900 кг

– Определённый класс: **HeavyDutyAGV ≡ AGV ⊓ (hasPayload ≥ 1000)**

– Ризонер сам относит agv_07 к HeavyDutyAGV — в фактах этого не записано

Пример

Диспетчер отбирает исполнителей для паллеты 900 кг обычным SPARQL-запросом и получает agv_07 — без единой строчки специального кода.

10 Диалог агентов: транспортировка task_3021

– Диспетчер → **cfp**: transport task_3021, pallet_512, zone_A → zone_D; :ontology warehouse

– agv_07 → **propose**: cost 42, eta 180 c; agv_11 → **refuse**: reason low-battery

– Диспетчер → **accept-proposal** для agv_07; исполнитель → **inform-done**

– conversation-id cnp-3021 связывает все сообщения одного диалога

– Исполнитель устроен по BDI: принятие предложения порождает намерение

Кейс

Заряд упал ниже порога на полпути: brf обновляет убеждения, reconsider пересматривает намерения, агент переключается на план зарядки и шлёт диспетчеру failure.

“

Новые типы задач и роботов вводятся добавлением аксиом, а не переписыванием кода.

Главный практический аргумент в пользу онтологий в MAPC

11 Онтологии в MAPC: выгоды и цена

Преимущества

- Интеграция компонентов разных производителей
- Гибкость: новая задача — новая аксиома
- Объяснимость BDI важна для сертификации
- Перенос опыта между агентами
- Семантическая навигация и понятный HRI

Ограничения

- Разработка онтологии трудоёмка и дорога
- Согласование онтологий не автоматизируется
- Вывод в выразительных профилях OWL дорог
- Вынос вывода в облако — зависимость от связи
- Заземление символов (symbol grounding)

Выводы

1. Агент — контур «сенсоры — контроллер — база знаний — актуаторы»; формально Ag: Per* → Ac
2. Реактивные архитектуры быстры и робастны, но живут настоящим; субсумпция Брукса — их эталон
3. BDI структурирует делиберацию через убеждения, желания и намерения; намерения инерционны
4. Гибриды (TouringMachines, InteRRaP) сочетают быстрый низ с планированием и кооперацией наверху
5. Онтология по Груберу — явная спецификация разделяемой концептуализации; фундамент — DL с TBox и ABox
6. Стек RDF → RDFS → OWL 2 → SPARQL; ризонеры HermiT, Pellet, FaCT++ дают новые знания из аксиом
7. FIPA ACL несёт перформатив и явную ссылку на онтологию; KnowRob и IEEE 1872 дают интероперабельность

Вопросы для самопроверки

1. Сформулируйте «слабое» понятие агентности; чем от него отличается «сильное»?

2. Запишите модели чисто реактивного агента и агента с внутренним состоянием

3. Что означают подавление и замещение в субсумпционной архитектуре Брукса?

4. Раскройте категории B, D, I; почему намерения должны обладать инерцией?

5. В чём дилемма стратегии reconsider? Когда выгоден «смелый» агент?

6. Сравните горизонтальную и вертикальную схемы; какие три уровня у InteRRaP?

7. Дайте определение онтологии по Груберу; чем TBox отличается от ABox?

8. Запишите в DL класс «мобильный робот с камерой, выполняющий инспекцию»

9. Какова структура сообщения FIPA ACL? Зачем нужны :ontology и :conversation-id?

10. Опишите Contract Net по шагам; что такое проблема заземления символов?